

Analysis: Fair and Square

The first thing to do in this problem (as in many other problems) is to make sure you read it carefully. Many contestants thought that 676 should be a fair and square number - after all, it is a square **and** a palindrome. It is *not*, however, a square **of** a palindrome, and this example is actually mentioned specifically in the problem statement!

The small input

Once you realize this, you can approach the small testcase by iterating over all the numbers Little John considers, and checking each one of them. You have to check for each number **X** whether it is a palindrome, and whether it is the square of a palindrome.

To check whether **X** is a palindrome, you can simply convert it to string format (the details here depend on the programming language you are using) and compare the first character to the last, the second to the second last, and so on.

To check whether **X** is the square of a palindrome, there are multiple options. One is to calculate the square root, and if the square root is an integer, check if it is a palindrome as described above. Another is simply to iterate over all numbers up to **X**, and for each palindrome, square it and see if the square is **X**. That's a perfectly good solution for the small input, but it will be too slow for the larger ones.

The first large input

For the first large input, we need to deal with numbers up to 10^{14} , and also with 10,000 test cases. A linear search of all numbers up to 10^{14} is not going to be fast enough, so we have to be smarter - we can't afford to check each number in the interval individually.

We don't really need to go all the way up to 10^{14} though! We are interested in numbers whose *squares* are Fair and Square and between **A** and **B** - and that means we have to check up to the *square root* of **B** only. That's only 10^7 numbers to check in the worst case.

We are not done though. While 10^7 numbers can be processed within the time limit, processing 10,000 cases like this is somewhat risky. There are two tricks you can notice to make your solution faster.

One trick is that we are interested not in all the numbers up to 10^7 , but only in palindromes. We can generate all the palindromes much faster. Start by taking all the numbers up to 10^4 , and then taking their mirror reflections (either duplicating the last number or not) to generate all palindromes of length up to 8 (and then square each and check whether it is a Fair and Square number in the interesting interval). This will cause us to evaluate only around 10,000 numbers for each test case, which is small enough that even a slow machine can deal with all the test cases in four minutes. You would need to use a reasonably efficient language however.

An alternative is to simply generate all the fair and square numbers up to 10^{14} *before* processing the test cases. There are relatively few of them — it turns out only 39. Thus, if you find all of them (in any fashion) before downloading the input file, you can easily give the correct answers to all the input cases.

Note that if you do this, you have to include the code you used to generate Fair and Square numbers - not just the code that includes the full list!

The second large data set

Now we come to the largest data set. Even combining both the tricks above is not enough - we need to go over 10^{25} palindromes to precompute everything. This will take a very long time in any language on any computer! A good idea here is to generate the first few Fair and Square numbers (and their square roots) to try and get an idea of what they look like. There are two things you can notice:

- All the Fair and Square numbers have an odd number of digits
- All the digits are rather small. In particular, with one exception, every square root of a Fair and Square number consists only of digits 0, 1 and 2.

Let's try to understand why these things would be true.

Let's begin with the "odd number of digits". A square of a number with N digits will have either $2N - 1$ or $2N$ digits, depending on whether there is a carry on the last position. Let's try to prove a carry never happens. Let X be Fair and Square, and let its square root be Y . Let the first digit of Y be c - then the first two digits of X are between c^2 and $(c+1)^2$. In particular:

- If the first digit of Y is 1, the first digit of X is between 1 and 4 - and thus no carry.
- If the first digit of Y is 2, the first digit of X is between 4 and 9 - and thus no carry.
- If the first digit of Y is 3, the first digits of X are between 9 and 16, so the first digit is 9 or 1. As Y is a palindrome, the last digit of Y is 3 as well, and thus the last digit of X is 9 - meaning the first digit of X is 9 as well, meaning no carry.
- If the first and last digit of Y is 4, the last digit of X is 6, while the first is either 1 or 2 - so X can't be Fair and Square.
- Similarly, if the first and last digit of Y is 5 (last digit of X is 5, first is 2 or 3), 6 (last digit of X is 6, first is 3 or 4), 7 (last digit of X is 9, first is 4, 5 or 6), 8 (last digit of X is 4, first is 6, 7 or 8) and 9 (last digit of X is 1, first is 8 or 9), then X also can't be Fair and Square.

This means there is no carry in the first digit.

Now since all the digits seem so small, maybe this means there is no carry at all? Note that if you take a palindrome and square it, and there's no carry, the result is a palindrome as well - so that would give us a nice characterization of Fair and Square numbers. Indeed, it turns out to be the case, and the proof follows.

Let Y have digits $(a_d)(a_{d-1})...(a_0)$. Let $b_k = a_0 * a_k + a_1 * a_{k-1} + ... + a_k * a_0$. Note that b_i is exactly the i th digit of $X = Y^2$ when performing long multiplication, before carries are performed. Since $a_j = a_{d-j}$, we also have $b_i = b_{2d-i}$.

Now suppose there's a carry in the long multiplication (meaning some b_j is greater than 9), and that we take a carry into digit i but no larger digits. We know digits i and $2d-i$ in X are equal, and are equal to b_i plus whatever we carried into digit i . Since we carry nothing to digit $i+1$, b_i is no larger than 9.

Now we will see that digit $2d-i$ of X has to equal b_{2d-i} (which is equal to b_i , and thus no larger than 9). For it to be different, we would have to carry something into digit $2d-i$ - but this would mean that b_j is larger than 9 for some $j < 2d-i$, and hence b_{2d-j} is also greater than 9 and we would have a carry after digit i .

Since X is a palindrome, this tells us that digit i of X is equal to b_i , which means that no carry entered digit i , and we have a contradiction.

We conclude that no carries were performed in the long multiplication at all.

Thus, the Fair and Square numbers are exactly the palindromes with no carries inside. In particular, the middle digit of **X** is the sum of squares of all the digits of **Y**, so this sum has to be no larger than nine. We conclude that only 0, 1, 2, and 3 can appear in **Y**.

To find all Fair and Square numbers, it therefore suffices to consider only palindromes consisting of these four digits, with the sum of squares of digits at most 9. It turns out this is a small enough set that it can be directly searched, allowing us to generate the full list of all Fair and Square numbers up to 10^{100} in a few seconds - and thus allowing us to solve the largest dataset.

Lessons learned

There are a few things we want to remind you of in the context of this problem:

- It is really important to read the problem statement carefully.
- We can sometimes have problems in which we don't have the standard combination of one small and one large input. The rules for dealing with small and large inputs are still the same (unless explicitly stated otherwise in the problem statement).
- We can also sometimes give problems that require very large integers. We did give Fair Warning about this some time ago, but it's always worth reminding.
- Finally, if you use precomputation in your solution, remember that you are required to provide us not only with the code that you actually used to solve the problem (containing the precomputed values), but also the code that you used for precomputation.

Analysis: Lawnmower

The Small Input

For a problem like this, it can be helpful to think through a few cases. Let's look at the first two examples from the problem statement:

2	2	2	2	2	
2	1	1	1	2	2
2	1	2	1	2	1
2	1	1	1	2	2
2	2	2	2	2	

All the grass needs to be cut to either height 1 or 2. Thus, we can begin by cutting the whole lawn to height 2. The question that remains is which rows and which columns to cut to height 1. Note that if we cut a row (or column) to height 1, all the squares in this row or column will be height 1 in the final pattern since we can't grow grass back.

In the left example, we must at some point do a cut with the lawnmower at height 1. Otherwise, we would never get any grass that low. However, there is nowhere that it is safe to make a cut like that. Every row and every column has at least one square where we want the final grass height to be 2, so we can never run the lawnmower through that row or column while at height 1. This pattern is impossible.

In the second example, we also must do some cuts with the lawnmower at height 1. However, in this case, there are two places where we can safely make that cut: the middle row and the middle column. If we do both, we get the desired pattern.

More generally, there are some rows and columns we cannot cut to height 1. By avoiding those rows and columns, we ensure nothing will be made too low. What remains is to check if it is still possible to get all the grass low *enough*. Well, if our only goal is to get the grass low, we should do all the cuts we can!

This suggests the following approach:

- Determine which rows and columns it is safe to cut at height 1 (meaning the pattern has no square with height > 1 in that row or column).
- Do a cut on each of these rows and columns at height 1.
- Check if we got every square low enough. If so, the pattern is possible. Otherwise, it is not.

The Large Input

For the large input, we can use almost the exact same strategy. You just have to think through what that means!

We can cut any row or column at a height that is equal to the maximum height appearing in this row (or column). As long as we follow this rule, we will never cut a square too low, and then as above, we just need to try to get everything low enough. For that purpose, we want to use all the cuts we can. The full algorithm is then:

- Iterate over every row and find the largest height that the pattern has in this row. Cut the row at this height.

- Do the same thing for every column.
- Output "YES" if this achieved the desired pattern, and "NO" if not.

Analysis: Tic-Tac-Toe-Tomek

In this problem, you had to classify the state of a Tic-Tac-Toe game with a twist. The board is 4x4, and an extra symbol can appear on the board - a "T", which either player can use for victory.

Note that you are guaranteed by the problem description the input will always describe a board that was obtained by a correct sequence of moves. As the game ends when one player wins, this guarantees that only one player can have four symbols (or 3 symbols and a T) in a completed line.

Thus, the simplest way to check whether a player, say "X", won is to check all rows, columns and diagonals whether they contain only "T"s and "X"s. Do not forget to check both diagonals! If there is a row, column or diagonal containing no "."s or "O"s, we know that X won. Similarly, if there is a row, column or diagonal containing no "X"s or "."s, O won.

If none of the players won, we only have to distinguish between a draw and a game not completed. This is relatively simple - if the board contains even a single ".", the game has not completed yet; otherwise it's a draw.

Note that your solutions are checked automatically, by a program. This means that your output has to be exactly matching the specification. A number of contestants had problems due to returning "O Won" instead of "O won" or "The game has not been completed" instead of "Game has not completed". In a programming competition it is important to follow the specification of the output as exactly as possible.

Here is a complete solution in Python for reference:

```
import sys

def solve(b):
    for c in ['X', 'O']:
        wind1 = True
        wind2 = True
        for x in range(4):
            winh = True
            winv = True
            for y in range(4):
                if b[y][x]!=c and b[y][x]!='T': winv = False
                if b[x][y]!=c and b[x][y]!='T': winh = False
            if winh or winv: return c + ' won'
            if b[x][x]!=c and b[x][x]!='T': wind1 = False
            if b[3-x][x]!=c and b[3-x][x]!='T': wind2 = False
        if wind1 or wind2: return c + ' won'

    for x in range(4):
        for y in range(4):
            if b[y][x]=='.': return 'Game has not completed'

    return 'Draw'

numcases = int(sys.stdin.readline())
```

```
for casenum in range(1,numcases+1):
    board = []
    for i in range(0,5):
        board.append(sys.stdin.readline().strip())
    print 'Case #' + repr(casenum) + ': ' + solve(board)
```

Analysis: Treasure

Lexicographically Smallest?

This problem asks you to open the chests in the lexicographically smallest order. As if it's not hard enough to find just any old way to open the chests, we make you find a particular way! If you have not tried similar problems before, this part might seem especially daunting.

However, it's a red herring. Suppose you can answer the following simpler question: "Is it possible at all to open all the chests?". Then you can use that as a black box to find the lexicographically smallest way of opening them. Check whether you have the key to open chest #1 first, and whether the black box says it is possible to open all the remaining chests after doing so. If the answer to both questions is yes, then you should definitely start by opening chest #1. Otherwise, you have no choice but to try a different chest. Repeating this logic for every chest at every time, you can get the lexicographically smallest solution.

This is not very fast, but there aren't too many chests to worry about here, so that's okay. And that means, from now on, we can focus on the slightly simpler question: "Is it possible to open all the chests?".

Eulerian Path

Unfortunately, even this question is still pretty hard!

When dealing with a really tricky problem, it can sometimes be helpful to look at special cases to build up your intuition. Towards that end, let's suppose that you begin with exactly one key, that one chest is empty, and that all remaining chests also contain exactly one key. Why look at this case in particular? You'll see!

In this version of the problem, you will always have exactly one key at any given time. When you open a chest of type A containing a key of type B, you are switching from a key of type A to a key of type B. This suggests that we can represent the problem as a graph. We will have one vertex for each chest/key type, and for every chest, we will add a directed edge from the chest type to the contained key type.

You begin with a single key, and then you must choose a chest of the matching type, giving you a key of perhaps a different type. From there, you must choose a chest of the new type, and so on, eventually choosing every chest exactly once. In the graph formulation, you can think of this as beginning at the vertex corresponding to your starting key, and then repeatedly following edges, using each edge exactly once. In other words, you are looking for an [Eulerian Path](#) on the graph!

The good news here is that Eulerian Path is a famous problem and you can look up on the internet how to tell if one exists:

- At most one vertex (namely the start vertex) has $\text{OutDegree} - \text{InDegree} = 1$.
- At most one vertex has $\text{InDegree} - \text{OutDegree} = 1$.
- All other vertices have $\text{InDegree} = \text{OutDegree}$.
- There is a path from the start vertex to every other vertex in the graph.

The bad news is that if this special case is already as hard as Eulerian Path, the full problem is going to be even harder!

Generalizing the Eulerian Path

If you come up with the previous observation, the first thing you might try is to reduce the full problem directly to Eulerian paths. Unfortunately, this is probably doomed to fail. Once you have multiple keys in a single chest, there is no graph structure to use. (Nothing we have found anyway!)

The better plan is to generalize only the conditions required for an Eulerian path to exist. In fact, those conditions can be described very naturally in terms of chests and keys:

- For each type, there must be at least as many keys of that type as there are chests of that type.
- It must be possible to get at least one key of any single type.

Let's say a chest/key configuration is *connected* if it satisfies the second condition, and *good* if it satisfies both conditions. It turns out that it is possible to open all the chests if and only if the configuration is good!

And it is not too hard to check if a configuration is good - the first condition is just a count, and the second can be checked with a [Breadth First Search](#) or a [Depth First Search](#). So all that remains for a complete solution is convincing ourselves that checking goodness is indeed equivalent to the original problem:

Claim: It is possible to open all the chests if and only if the configuration is good.

Proof: One direction is easy. If the configuration is not good, then we will never be able to open enough chests of one type, either because there are not enough keys, or because we can never reach even one of those keys.

For the other direction, let's suppose we have a good configuration. Nothing we do from this point on will change whether there are enough keys of each type. We will show there is always at least one chest we can open that also maintains the connectivity property. The resulting configuration would then also be good, so there would also be a chest we could open there that would maintain connectivity, and so on. Repeating, we can keep opening chests until there is nothing left.

So all we need to do is prove that there is at least one chest that can be opened without breaking connectivity. We know the configuration is connected initially. For each type T , there is a sequence of chests we can open to get a key of type T . To be precise, here exists a sequence of types $T_1, T_2, \dots, T_n$, with the following properties:

- You already have at least one key of type T_1 .
- $T_n = T$.
- For each i , there is a chest of type T_i which you would be able to open to get a key of type T_{i+1} .

Suppose you have a key of some arbitrary type A . If you already have *all* keys of type A , then you can open all the chests of type A , and doing so will certainly not break connectivity. So that case is easy.

Otherwise, there must exist some chest of type B containing a key of type A. Let $T_1, T_2, \dots, T_{n-1}=B, T_n=A$ denote a sequence of key types that you can go through to get a key of type B and then use that to get another key of type A. As mentioned above, we know such a sequence must exist. We can also assume that $T_i \neq A$ for $1 < i < n$. Otherwise, we could just chop off part of the sequence to get a faster method! Now we consider two cases:

- Suppose that $T_1 \neq A$. Then you can use your key of type A to open anything you want, and we claim the resulting configuration will still be connected.

To prove this, pick an arbitrary key type C (possibly equal to A). Before opening any chests, we know there is some sequence of key types $S_1, S_2, \dots, S_m=C$ that will give you a key of type C. If A is not part of this list, then opening a chest of type A does not interfere with getting a key of type C. Otherwise, the S list and T list intersect somewhere, so we can let j be the largest integer such that S_j equals some T_i . Then after using our A key, we can still get a key of type C in the following way: $T_1, T_2, \dots, T_i=S_j, S_{j+1}, \dots, S_m=C$.

Since this is true for every C, we know the configuration is still connected!

- Suppose that $T_1 = A$. In this case, you should use your key to open a chest of type T_2 . You can now still obtain a key of type B, and so the rest of the argument follows exactly as before.

Therefore, no matter what happens, you can always open a chest without breaking connectivity, and the claim is proven!